ChatGPT

Automated EDA and ML Pipeline Script

We can build a Python script that loads any CSV file and performs automated exploratory data analysis (EDA) and supervised learning. The script uses common data libraries (e.g. **pandas**, **numpy**) to read and preprocess the data, visualization libraries (e.g. **matplotlib**, **seaborn**) to plot charts, and machine learning libraries (scikit-learn) for modeling. Several automated EDA tools exist – for example **pandas-profiling**, **Sweetviz**, and **AutoViz** can each generate a comprehensive EDA report with a single command [1]. Other packages like **Missingno** help visualize missing data, and GUI libraries like **D-Tale** or **PandasGUI** allow interactive browsing of a DataFrame.

- **Data Loading:** Use `pandas.read_csv()` to read the user's CSV file into a `DataFrame`. Pandas is the standard library for tabular data and handles CSV import easily.
- **Basic Inspection:** Print the dataset's shape, data types, and head/tail. Compute summary statistics with `df.describe()` and show null-counts via `df.isnull().sum()`. (Missingno can also draw a *matrix* or *heatmap* of missing values [2].)
- **Automated Reports:** Optionally call tools like Pandas Profiling or Sweetviz to produce an HTML report. For example, `pandas_profiling.ProfileReport(df)` or `sv.analyze(df)` will compute distributions, correlations, and give an "at-a-glance" summary of every column [1]. These reports include histograms, bar charts, and correlation matrices for numeric and categorical fields.

- **Custom Plots:** In addition, the script can loop over numeric columns and plot histograms/boxplots, and for categorical columns plot bar charts. For instance, using Matplotlib/Seaborn one might do: `plt.hist(df[col])` or `sns.boxplot(data=df, x=col)`. Numeric distributions often reveal skew or outliers; an example histogram is shown below.

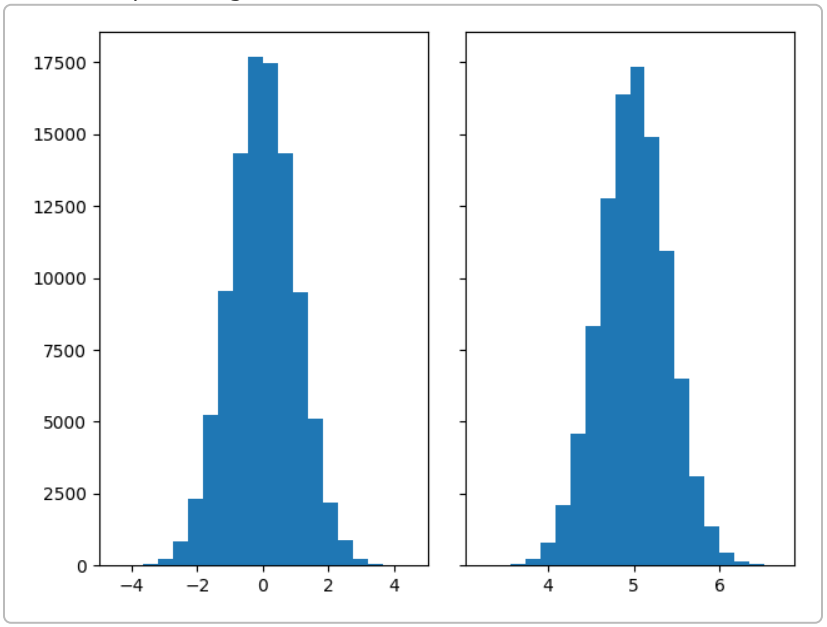

Figure: Example histograms of two numeric features (created with Matplotlib). The script can generate similar plots for each numeric column.

- **Correlation:** Compute and plot a correlation matrix (e.g. `sns.heatmap(df.corr(), annot=True)`) to identify strong linear relationships. If many features exist, a heatmap can highlight which pairs are highly correlated.

- **Interactive GUI:** To allow the user to view data in a spreadsheet-like interface, one can use **PandasGUI** or **D-Tale**. PandasGUI (for example) opens a GUI window where you can filter, sort, and plot data by drag-and-drop [3]. It can also show summary statistics and allow interactive plotting without writing code [3] [4]. For instance:

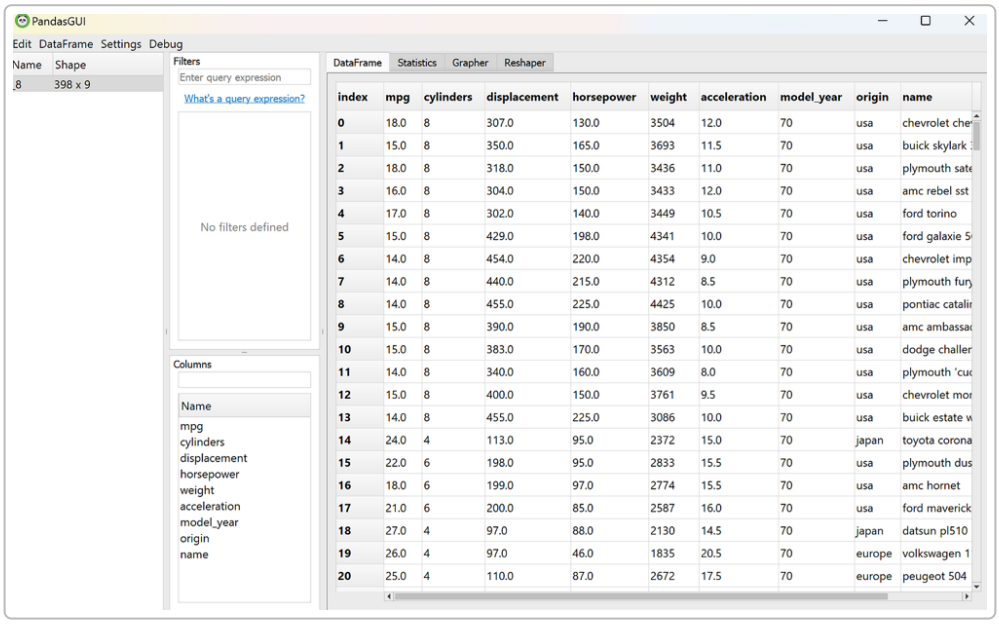

Figure: PandasGUI launched on a DataFrame (screenshot). It provides tabs for data, statistics, and plotting [3] [4].

- **Supervised Learning:** Once EDA is done, the script can proceed to train a model. Typically you identify one column as the *target*. If it is categorical (few unique values), treat it as a classification problem; if numeric, as regression. We then split the data into training and test sets – for example using `sklearn.model_selection.train_test_split()` [5]. This separation is crucial for unbiased evaluation of the model [5].

- **Model Training:** Using scikit-learn, the script can train several algorithms automatically. Scikit-learn provides many supervised algorithms (e.g. logistic regression, decision trees, random forests, etc.) that can be tried in turn [6]. For example, if `y` is categorical, we might train `RandomForestClassifier()` or `LogisticRegression()` on `X_train, y_train` and compute accuracy on the test set. If `y` is continuous, a regressor like `RandomForestRegressor()` or `LinearRegression()` can be used and we report RMSE or R^2. Metrics (accuracy, precision/recall, or MSE) and plots (confusion matrix for classification, true-vs-prediction scatter for regression) can be output.

- **Automation (AutoML):** For even less manual work, AutoML libraries (Auto-Sklearn, TPOT, etc.) can be integrated. These tools automatically search over model types and hyperparameters to find a

good pipeline [7]. In practice, one could either call an AutoML routine, or simply try a handful of models in a loop. The idea is that "the user simply provides data, and the AutoML system automatically determines the approach that performs best" [7].

- **Command-line interface:** The script can be run from the terminal. It could use `argparse` to accept the CSV file path as an argument, and then prompt for actions (e.g. which column to predict). After processing, it can open matplotlib windows (`plt.show()`) for each chart or call PandasGUI's `show(df)` to display the interactive table.

Below is a **schematic Python script** illustrating these steps in a modular way. It uses pandas for data I/O, matplotlib/seaborn for plots, scikit-learn for modeling, and PandasGUI for an optional GUI. (Comments explain each section.)

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from pandasgui import show
import missingno as msno
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier, RandomForestRegressor
from sklearn.linear_model import LogisticRegression, LinearRegression
from sklearn.metrics import accuracy_score, mean_squared_error,
classification_report, confusion_matrix

def load_data(path):
    """Load CSV into a DataFrame."""
    return pd.read_csv(path)

def basic_eda(df):
    """Perform basic EDA: print info, plot distributions."""
    print("Shape:", df.shape)
    print("\nData Types:\n", df.dtypes)
    print("\nMissing values:\n", df.isnull().sum())
    print("\nStatistics:\n", df.describe(include='all'))
    # Plot histograms for numeric columns
    num_cols = df.select_dtypes(include=np.number).columns
    for col in num_cols:
        plt.figure()
        df[col].hist(bins=20)
        plt.title(f"Histogram of {col}")
        plt.xlabel(col)
        plt.ylabel("Count")
    plt.show()
    # Plot missing-value matrix (optional)
    try:
```

```
        msno.matrix(df)
        plt.show()
    except Exception as e:
        print("Missingno plotting failed:", e)
    # Correlation heatmap
    plt.figure(figsize=(6,5))
    sns.heatmap(df.corr(), annot=True, cmap='coolwarm')
    plt.title("Correlation matrix")
    plt.show()

def train_and_evaluate(df, target):
    """Train a model on the target column and print evaluation."""
    X = df.drop(columns=[target])
    y = df[target]
    # Simple encoding: convert categorical to codes
    for col in X.select_dtypes(include=['object','category']).columns:
        X[col] = X[col].astype('category').cat.codes
    # Split into train/test to avoid overfitting 5
    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.2, random_state=42)
    # Choose model based on problem type
    if y.dtype == 'object' or str(y.dtype).startswith('category') or
len(np.unique(y)) < 10:
        print("Classification problem detected.")
        model = RandomForestClassifier(random_state=42)
        model.fit(X_train, y_train)
        pred = model.predict(X_test)
        print("Accuracy:", accuracy_score(y_test, pred))
        print(classification_report(y_test, pred))
        # Plot confusion matrix
        cm = confusion_matrix(y_test, pred)
        sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
        plt.title("Confusion Matrix")
        plt.xlabel("Predicted")
        plt.ylabel("True")
        plt.show()
    else:
        print("Regression problem detected.")
        model = RandomForestRegressor(random_state=42)
        model.fit(X_train, y_train)
        pred = model.predict(X_test)
        rmse = np.sqrt(mean_squared_error(y_test, pred))
        print(f"Test RMSE: {rmse:.3f}")
        # Plot true vs predicted
        plt.scatter(y_test, pred, alpha=0.6)
        plt.xlabel("True Values")
        plt.ylabel("Predictions")
        plt.title("True vs Predicted")
```

```
        plt.show()

def main():
    import argparse
    parser = argparse.ArgumentParser(description="Automated EDA and ML script")
    parser.add_argument('csvfile', help="Path to input CSV file")
    args = parser.parse_args()

    df = load_data(args.csvfile)
    basic_eda(df)  # perform exploratory analysis
    # Ask user for target column
    target = input("Enter the name of the target column for prediction: ")
    if target in df.columns:
        train_and_evaluate(df, target)
    else:
        print(f"Column '{target}' not found in data.")

    # Optionally launch an interactive GUI (PandasGUI) to view data
    if input("Open data GUI? (y/n): ").lower() == 'y':
        show(df)  # opens a window with the DataFrame [3]

if __name__ == "__main__":
    main()
```

Explanation of key parts:

1. **Loading and Preprocessing:** We read the CSV with `pd.read_csv`. We then encode categorical features as needed (the code above simply converts them to numerical codes).
2. **EDA:** The function `basic_eda()` prints data info and draws plots. We use `matplotlib` / `seaborn` to plot histograms and a heatmap of correlations. The missing-value matrix (via Missingno) and histograms help identify outliers or skewness.
3. **Model Training:** The function `train_and_evaluate()` uses `sklearn.model_selection.train_test_split` to split data into train and test sets [5]. We detect if the target is categorical (string or low cardinality) to choose classification vs regression. Then we fit a Random Forest (as an example) and print performance metrics. Accuracy and a confusion matrix are shown for classification; RMSE and a scatter plot are shown for regression. (Many other models could be tried similarly.) This process follows standard supervised learning practice [6] [5].
4. **Automated Tools:** The code also mentions PandasGUI's `show(df)` which opens an interactive DataFrame explorer (see above image). This lets the user sort, filter, and even plot columns by drag-and-drop [3] [4]. Similarly, one could integrate packages like **D-Tale** which launch a web-app interface for the DataFrame [8].
5. **Putting It All Together:** We tie these parts in the `main()` function. The script uses `argparse` to accept the CSV filename, runs EDA, then asks for a target, then trains a model. Charts (`plt.show()`) will pop up in separate windows as the script runs. Finally, it can open an interactive GUI for further data exploration.

This modular script framework can handle *any* tabular CSV: it automatically computes summaries, makes standard plots, splits the data, trains a model, and presents results. By using libraries like pandas,

matplotlib, and scikit-learn (all widely-used tools for data science [6] [5]), as well as optional GUI libraries (PandasGUI [3], D-Tale [8]) and automated EDA/reporting tools [1], the script provides a one-stop solution for Exploratory Data Analysis and supervised learning on arbitrary datasets.

Sources: We rely on best practices from the Python data ecosystem. For example, tools like Pandas Profiling, Sweetviz, and AutoViz are documented for one-line EDA reports [1], and libraries like PandasGUI provide interactive data viewing [3] [4]. The machine learning portion uses standard scikit-learn methods as described in the documentation and tutorials [6] [5]. These references validate the use of these libraries and methods in an automated EDA/ML pipeline.

---

[1] Automated EDA and Creating a Simple Interactive Dashboard in Python | by Mutiara Tambunan | Medium
https://medium.com/@muteknii/automated-eda-and-creating-a-simple-interactive-dashboard-in-python-7789752f220b

[2] [4] [8] Top Automated EDA Python Packages for Efficient Data Analysis
https://www.nb-data.com/p/python-packages-for-automated-eda

[3] GitHub - adamerose/PandasGUI: A GUI for Pandas DataFrames · GitHub
https://github.com/adamerose/PandasGUI

[5] Split Your Dataset With scikit-learn's train_test_split() – Real Python
https://realpython.com/train-test-split-python-data/

[6] Comprehensive Guide to Classification Models in Scikit-Learn - GeeksforGeeks
https://www.geeksforgeeks.org/machine-learning/comprehensive-guide-to-classification-models-in-scikit-learn/

[7] Automated Machine Learning (AutoML) Libraries for Python - MachineLearningMastery.com
https://machinelearningmastery.com/automl-libraries-for-python/